

DSA FINAL PROJECT



Submitted By:

Huzaifa Adil Akram	22024119-054
Syeda Qandeel Fakhar	22024119-004
Nimra Jamsheed	22024119-015
Haroon Ali Imran	22024115-027
Syeda Fatima Qasim	22024119-028

Submitted To:

Mr. Zulfiqar

BS (Computer Science)

2022-2024

Date: 9th January 9, 2024

FACULTY OF COMPUTING & IT

This program uses CSV file name “prog_book.csv” which I took from Kaggle to load book data into the Linked List Nodes.

HEADER FILE:
BorrowHistorystack.h

```
#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;
struct Node {
    string BookName;
    string StudentName;
    Node* next;
};
class stack {
public:
    Node *stacktop;
    stack(){
        stacktop = NULL;
    }
    bool empty(){
        return stacktop == NULL;
    }
    void push(string stuName, string bookName){
        Node *temp = new Node;
        temp->StudentName = stuName;
        temp->BookName = bookName;
        temp->next = NULL;
        if (empty()){
            stacktop = temp;
            return;
        }
        temp->next = stacktop;
        stacktop = temp;
    }
    string pop(){
        Node *temp = stacktop;
        stacktop = stacktop->next;
        string deletedBook = temp->BookName;
        delete temp;
        return deletedBook;
    }
    void display(){
        Node *cur = stacktop;
        while (cur!=NULL)
        {
            cout << setw(15) << left << cur->StudentName << "|";
            cout << setw(15) << left << cur->BookName << endl;
            cur = cur->next;
        }
    }
    string top(){
        return (stacktop->BookName);
    }
}
```

```

void saveBookToFile(string stuName, string bookName){
    ofstream outputFile;
    outputFile.open("history.txt", ios::app);
    if (!outputFile)
    {
        cerr << "Error opening file: history.txt" << endl;
        return;
    }
    outputFile << stuName << "," << bookName << endl;
    outputFile.close();
}

void loadStack(){
    ifstream inputFile("history.txt");
    if (!inputFile)
    {
        cerr << "Error opening file: history.txt" << endl;
        return;
    }
    string line;
    while (getline(inputFile, line))
    {
        stringstream ss(line);
        string NewStudentName;
        string NewBookName;
        getline(ss, NewStudentName, ',');
        getline(ss, NewBookName, ',');
        push(NewStudentName, NewBookName);
    }
    inputFile.close();
}

// Function to update the history file after popping a book
void updateHistoryFile(stack& historyStack) {
    ofstream outputFile("history.txt");
    if (!outputFile) {
        cerr << "Error opening file: history.txt" << endl;
        return;
    }

    Node* cur = historyStack.stacktop;
    while (cur != NULL) {
        outputFile << cur->StudentName << "," << cur->BookName << endl;
        cur = cur->next;
    }

    outputFile.close();
}

};

```

Main Program:
main.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <iomanip>

#include "BorrowHistoryStack.h"    //personal header file.

using namespace std;

struct BookNode
{
    string title;
    int pages;
    float rating;
    string status;

    BookNode *next;
};

class BookList
{
    BookNode *head, *tail;

public:
    BookList() : head(NULL), tail(NULL) {}

    int countNodes(){
        int count = 0;
        BookNode* cur = head;
        while (cur != NULL)
        {
            count++;
            cur = cur->next;
        }
        return count;
    }

    bool isEmpty(){
        if (head == NULL){
            return true;
        } else {
            return false;
        }
    }

    string insert_start(BookNode* temp){
        if (head == NULL)
        {
            head = tail = temp;
            return temp->title;
        }
        temp->next = head;
        head = temp;
        return head->title;
    }
}
```

```

string insert_end(BookNode* temp){
    if (head == NULL)
    {
        head = tail = temp;
        return temp->title;
    }
    tail->next = temp;
    tail = temp;
    return tail->title;
}

string insert_position(BookNode* temp, int pos){
    if (pos == 1)
    {
        return insert_start(temp);
    }
    else if (pos == countNodes())
    {
        return insert_end(temp);
    }
    else if (pos > 1 && pos <= countNodes()) {
        BookNode *cur,*prev;
        cur = head;
        for (int i = 1; i < pos; i++){
            prev = cur;
            cur = cur->next;
        }
        prev->next = temp;
        temp->next = cur;
        return temp->title;
    } else {
        cout << "Invalid Position" <<endl;
        cin.ignore();
        cin.get();
        return "INVALID";
    }
}

string delete_start(){
    string bookname = head->title;
    BookNode* temp = head;
    head = head->next;
    delete temp;
    return bookname;
}

string delete_end(){
    string bookname = tail->title;
    BookNode *cur, *prev;
    cur = head;
    for (int i = 1; i < countNodes(); i++)
    {
        prev = cur;
        cur = cur->next;
    }
    tail = prev;
    tail->next = NULL;
    delete cur;
    return bookname;
}

```

```

string delete_at_position(int position){
    if (isEmpty())
    {
        cout << "There is no Book!\n";
        return "INVALID";
    }
    else if (position == 1)
    {
        return delete_start();
    }
    else if (position == countNodes())
    {
        return delete_end();
    }
    else if ((position > 1) && (position < countNodes())){
        string bookname;
        BookNode* cur, *prev;
        cur = head;
        for (int i = 1; i < position; i++)
        {
            prev = cur;
            cur = cur->next;
        }
        prev->next = cur->next;
        bookname = cur->title;
        delete cur;
        return bookname;
    }
    else{
        cout << "INVALID POSITION!\n";
        cin.ignore();
        cin.get();
        return "INVALID";
    }
}

void addBook(){
    system("cls");
    BookNode* NewBook = new BookNode;
    int position;
    string NewBookTitle;
    NewBook->next = NULL;

    cout << "*****\t PLEASE PROVIDE INFORMATION TO ADD NEW BOOK\t*****\n";
    cout << "ENTER BOOK NAME: ";
    cin.ignore();
    getline(cin, NewBook->title);
    cout << "ENTER NUMBER OF PAGES: ";
    cin >> NewBook->pages;
    cout << "ENTER BOOK RATING: ";
    cin >> NewBook->rating;
    NewBook->status = "Available";
    cout << "There are " << countNodes() << " books.";
    cout << "\nWhere you want to place this book? Enter Position: ";
    cin >> position;
    NewBookTitle = insert_position(NewBook, position);

    if(NewBookTitle != "INVALID"){
        system("cls");
        cout << "*****";
    }
}

```

```

        cout << "\n\" << NewBookTitle << "\"";
        cout << " HAS BEEN ADDED SCCUSSFULLY!\n";
        cout << "PRESS ENTER KEY";
        cin.ignore();
        cin.get();
    }
}

void searchBook(bool management) {
    string bookname;
    BookNode* cur;
    system("cls");
    cout << "*****\tSTRICT SEARCH ENGINE\t*****\n";
    cout << "ENTER BOOK NAME: ";
    cin.ignore();
    getline(cin, bookname);

    cur = head;
    int count = 1;
    bool found = false;

    while (cur != NULL) {
        if (cur->title == bookname) {
            displayBookDetails(*cur);
            if(management){
                cout << "\nDo you want to delete this book? (y/n): ";
                char choice;
                cin >> choice;
                if (choice == 'y' || choice == 'Y') {
                    string deletedBook = delete_at_position(count);
                    if (deletedBook != "INVALID") {
                        system("cls");
                        cout << "\n*****\n";
                        cout << "\" << deletedBook << "\" HAS BEEN DELETED SCCUSSFULLY!\n";
                    }
                }
            }
            found = true;
            break; // Exit loop once the book is found
        }

        cur = cur->next;
        count++;
    }

    if (!found) {
        cout << "Book not found!\n";
    }

    cout << "Press any key to go back...";
    cin.ignore();
    cin.get();
}

void deleteBook(){
    int position;
    string deletedBook;
    BookNode *cur, *prev;
    displayBooks();
    cout << "\nWhich book do you want to delete? Enter Position: ";
    cin >> position;

```

```

deletedBook = delete_at_position(position);
if (deletedBook != "INVALID")
{
    system("cls");
    cout << "*****\n";
    cout << "\"" << deletedBook << "\"";
    cout << " HAS BEEN DELETED SCCUSSFULLY!\n";
    cout << "PRESS ENTER KEY";
    cin.ignore();
    cin.get();
}
}

void displayBookDetails(const BookNode &book)
{
    system("cls");
    cout << "Title: " << book.title << endl;
    cout << "Pages: " << book.pages << endl;
    cout << "Rating: " << book.rating << endl;
    cout << "Status: " << book.status << endl;
    cout << "-----" << endl;
}

void displayBooks()
{
    system("cls");
    BookNode *current = head;
    int count = 1;

    cout << "Books in the Linked List:" << endl;
    while (current != NULL)
    {
        cout << setw(3) << left << count << ". " << setw(45) << left << current->title;
        count++;
        current = current->next;

        if (current != NULL)
        {
            cout << setw(3) << left << count << ". " << current->title << "\n";
            count++;
            current = current->next;
        }
    }
}

void displayBookOption(){
    displayBooks();
    int choice;
    cout << "\nEnter the number of the book to view details (0 to go back): ";
    cin >> choice;

    if (choice == 0)
    {
        return; // Go back to the main menu
    }

    BookNode* current = head;
    int count = 1;
    while (current && (count < choice))
    {
        current = current->next;
        count++;
    }
}

```

```

    if (current)
    {
        cout << "\nDetails of the selected book:\n";
        displayBookDetails(*current);
        cout << "Press any key to go back...";
        cin.ignore();
        cin.get();    // Wait for user input to go back
    }
    else
    {
        cout << "\nInvalid choice. Please enter a valid number.\n";
        cout << "Press any key to continue...";
        cin.ignore();
        cin.get();    // Wait for user input to go back
    }
}

void borrowBook(const string& studentName, stack& myHistory) {
    displayBooks(); // Display all books

    int choice;
    cout << "\nEnter the number of the book to borrow (0 to go back): ";
    cin >> choice;

    if (choice == 0) {
        return; // Go back
    }

    BookNode* current = head;
    int count = 1;

    while (current && (count < choice)) {
        current = current->next;
        count++;
    }
    system("cls");
    if (current) {
        if (current->status == "Available") {
            cout << "\nThanks for borrowing \"" << current->title << "\"!\n";
            // Update the book status to "Borrowed"
            current->status = "Borrowed";
            myHistory.push(studentName, current->title);
            myHistory.saveBookToFile(studentName, current->title);
        } else {
            cout << "\nSorry, \"" << current->title << "\" is not available for borrow.\n";
        }
    } else {
        cout << "\nInvalid choice. Please enter a valid number.\n";
    }
    cout << "Press any key to go back...";
    cin.ignore();
    cin.get();
}

void returnBook(stack& historyStack, BookList& bookList) {
    system("cls");
    // Check if the stack is empty
    if (historyStack.empty()) {
        cout << "No book to return!\n";
        cout << "Press any key to go back...";
        cin.ignore();
    }
}

```

```

        cin.get();
        return;
    }

    // Get the book from the top of the stack
    string returnedBook = historyStack.pop();
    historyStack.updateHistoryFile(historyStack);

    // Find the book in the linked list and update its status to "Available"
    BookNode* current = bookList.head;
    while (current != NULL) {
        if (current->title == returnedBook) {
            current->status = "Available";
            break;
        }
        current = current->next;
    }

    // Display success message
    cout << "Book \" " << returnedBook << "\" has been returned successfully!\n";
    cout << "Press any key to go back...";
    cin.ignore();
    cin.get();
}

void selectionSortByRating(BookList& bookList) {
    BookNode* current = bookList.head;

    while (current != NULL) {
        BookNode* minNode = current;
        BookNode* nextNode = current->next;

        // Find the minimum node in the remaining unsorted part
        while (nextNode != NULL) {
            if (nextNode->rating < minNode->rating) {
                minNode = nextNode;
            }
            nextNode = nextNode->next;
        }
        // Swap the data, not just the pointers
        std::swap(current->title, minNode->title);
        std::swap(current->pages, minNode->pages);
        std::swap(current->rating, minNode->rating);
        std::swap(current->status, minNode->status);
        current = current->next; // Move to the next unsorted element
    }
    system("cls");
    cout << "BOOKS HAS BEEN SORTED ACCORDING TO RATING...\n";
    cout << "Press any key to go back...";
    cin.ignore();
    cin.get();
}

void checkBorrowHistory(stack& history){
    system("cls");
    cout << "*****\tBORROW HISTORY\t*****\n";
    cout << setw(15) << left << "STUDENT NAME" << "|" << setw(10) << left << "BOOK NAME" << endl;
    history.display();
    cout << "*****\n";
    cout << "Press any key to go back...";
    cin.ignore();
}

```

```

        cin.get();
    }
    void readCSVFile(const string &filename)
    {
        ifstream file(filename);
        if (!file)
        {
            cerr << "Error opening file: " << filename << endl;
            return;
        }

        string line;
        getline(file, line); // Skip the header line

        while (getline(file, line))
        {
            stringstream ss(line);
            BookNode *newBook = new BookNode;

            getline(ss, newBook->title, ',');
            ss >> newBook->pages;
            ss.ignore();
            ss >> newBook->rating;
            ss.ignore();
            getline(ss, newBook->status, ',');

            newBook->next = NULL;

            insert_end(newBook);
        }
        file.close();
    }
    void updateBookCSV(const BookList& bookList, const string& filename) {
        ofstream outputFile(filename);
        if (!outputFile) {
            std::cerr << "Error opening file: " << filename << std::endl;
            return;
        }

        // Write the header to the CSV file
        outputFile << "Book_title,Number_Of_Pages,Rating,Status" << std::endl;

        // Write each book's information to the CSV file
        BookNode* current = bookList.head;
        while (current != NULL) {
            outputFile << current->title << "," << current->pages << ","
                << current->rating << "," << current->status << endl;
            current = current->next;
        }
        outputFile.close();
    }
};

void register_account(string filename)
{
    system("cls");
    string uName, pass, email, gender;
    string valname, valpass;
    int phone;
    ofstream output;
    output.open(filename, ios::app);

```

```

    cout << "*****ACCOUNT REGISTRATION*****\n";
    cout << "ENTER USERNAME: ";
    cin >> uName;
    cout << "ENTER PASSWORD: ";
    cin >> pass;
    cout << "ENTER EMAIL: ";
    cin >> email;
    cout << "ENTER GENDER: ";
    cin >> gender;

    output << uName << " " << pass << " " << email << " " << gender << " " << phone << endl;
    output.close();
    system("cls");
    cout << "\n*****";
    cout << "\nYOUR ACCOUNT HAVR BEEN REGISTERED!" << endl;
    cout << "YOU CAN NOW LOGIN WITH YOUR ID AND PASSWORD" << endl;
    cout << "PRESS ENTER" << endl;
    cin.ignore();
    cin.get();
}

bool login_account(string filename, string &loggedStudentName) {
    system("cls");
    string uName, pass;
    string logname, logpass;
    ifstream input(filename);

    if (!input) {
        cout << "Error opening file!" << endl;
        return false;
    }
    cout << "*****PLEASE ENTER YOUR LOGIN INFORMATION*****\n";
    cout << "Enter username:" << endl;
    cin >> logname;
    cout << "Enter password:" << endl;
    cin >> logpass;

    while (input >> uName >> pass) {
        if (uName == logname && pass == logpass) {
            cout << "Login successful" << endl;
            loggedStudentName = logname;
            input.close();
            return true;
        }
    }

    cout << "Login failed!" << endl;
    input.close();
    return false;
}

int main()
{
    int choice;
    bool loggedIN;
    string currentStudentName;
    string currentManagerName;
    bool isManagement = false;
    bool isStudent = false;

    BookList Mylibrary;

```

```

Mylibrary.readCSVFile("prog_book.csv");

stack history;
history.loadStack();

while (choice != 3)
{
    system("cls");
    loggedIN = false;
    isManagement = false;
    isStudent = false;

    cout << "*****\n";
    cout << "****\tLibrary Management System\t****" << endl;
    cout << "*****\n";
    cout << "1.Management\n";
    cout << "2.Student\n";
    cout << "3.EXIT\n";
    cout << "*****\n";
    cout << "ENTER YOUR CHOICE: ";
    cin >> choice;

    if (choice == 1)
    {
        system("cls");
        int managementchoice;
        cout << "*****\n";
        cout << "****\t\tMANAGEMENT PORTAL\t****\n";
        cout << "*****\n";
        cout << "1.Login\n";
        cout << "*****\n";
        cout << "ENTER YOUR CHOICE: ";
        cin >> managementchoice;
        if (managementchoice == 1)
            loggedIN = login_account("managment.txt", currentManagerName);
        if (loggedIN)
        {
            isManagement = true;
        }
    }
    else if (choice == 2)
    {
        int studentChoice;
        system("cls");
        cout << "*****\n";
        cout << "****\t\tSTUDENT PORTAL\t\t****" << endl;
        cout << "*****\n";
        cout << "1.Register\n";
        cout << "2.Login\n";
        cout << "*****\n";
        cout << "ENTER YOUR CHOICE: ";
        cin >> studentChoice;
        switch (studentChoice)
        {
            case 1:
                register_account("student.txt");
                break;
            case 2:
                loggedIN = login_account("student.txt", currentStudentName);
                if (loggedIN)
                {

```

```

        isStudent = true;
    }
    break;
default:
    break;
}
}
else if (choice != 3)
{
    cout << "Please enter a valid choice!" << endl;
}

if (loggedIN)
{
    while (true)
    {
        system("cls");
        cout << "\nWelcome ";
        if (isManagement)
        {
            cout << "Management";
            // Display management-specific options
            cout << "\n1. Delete a book\n";
            cout << "2. Add a new book\n";
            cout << "3. Search for a book\n";
            cout << "4. Display all books\n";
            cout << "5. Check borrow history\n";
            cout << "6. Sort books by rating\n";
            cout << "7. Logout\n";
            cout << "8. EXIT\n";
            cin >> choice;

            if (choice == 1)
            {
                Mylibrary.deleteBook(); // Logout
            }
            else if (choice == 2)
            {
                Mylibrary.addBook();
            }
            else if (choice == 3)
            {
                Mylibrary.searchBook(true);
            }
            else if (choice == 4)
            {
                Mylibrary.displayBookOption();
            }
            else if (choice == 5)
            {
                Mylibrary.checkBorrowHistory(history);
            }
            else if (choice == 6)
            {
                Mylibrary.selectionSortByRating(Mylibrary);
            }

            else if (choice == 7)
            {
                Mylibrary.updateBookCSV(Mylibrary, "prog_book.csv");
                break;
            }
        }
    }
}

```

```

    }
    else if (choice == 8)
    {
        Mylibrary.updateBookCSV(Mylibrary, "prog_book.csv");
        return 0;    // Exit
    }
}
else if (isStudent)
{
    cout << "Student";
    // Display student-specific options
    cout << "\n1. Borrow a book\n";
    cout << "2. Return a book\n";
    cout << "3. Search a book\n";
    cout << "4. Display all books\n";
    cout << "5. Logout\n";
    cout << "6. EXIT\n";
    cin >> choice;

    if (choice == 1)
    {
        Mylibrary.borrowBook(currentStudentName, history);
    }
    else if (choice == 2)
    {
        Mylibrary.returnBook(history, Mylibrary);
    }
    else if (choice == 3)
    {
        Mylibrary.searchBook(false);
    }
    else if (choice == 4)
    {
        Mylibrary.displayBookOption();
    }
    else if (choice == 5)
    {
        Mylibrary.updateBookCSV(Mylibrary, "prog_book.csv");
        break; // Logout
    }
    else if (choice == 6)
    {
        Mylibrary.updateBookCSV(Mylibrary, "prog_book.csv");
        return 0; // Exit
    }
}
}
}
}
return 0;
}

```

Output:

Main Interface:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

*****
****      Library Management System      ****
*****

1.Management
2.Student
3.EXIT
*****
ENTER YOUR CHOICE:
```

Management Portion:

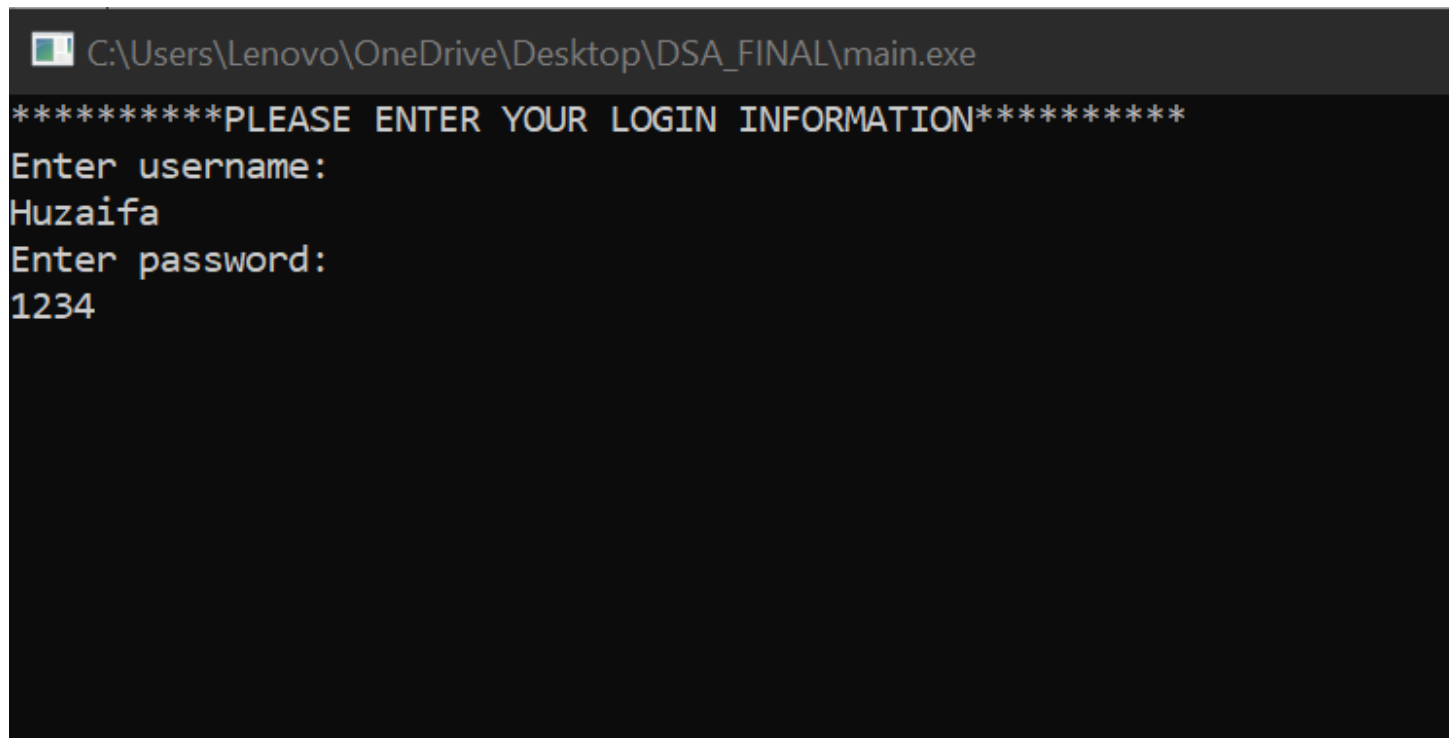
LOGIN:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

*****
*****      MANAGEMENT PORTAL      *****
*****

1.Login
*****
ENTER YOUR CHOICE:
```

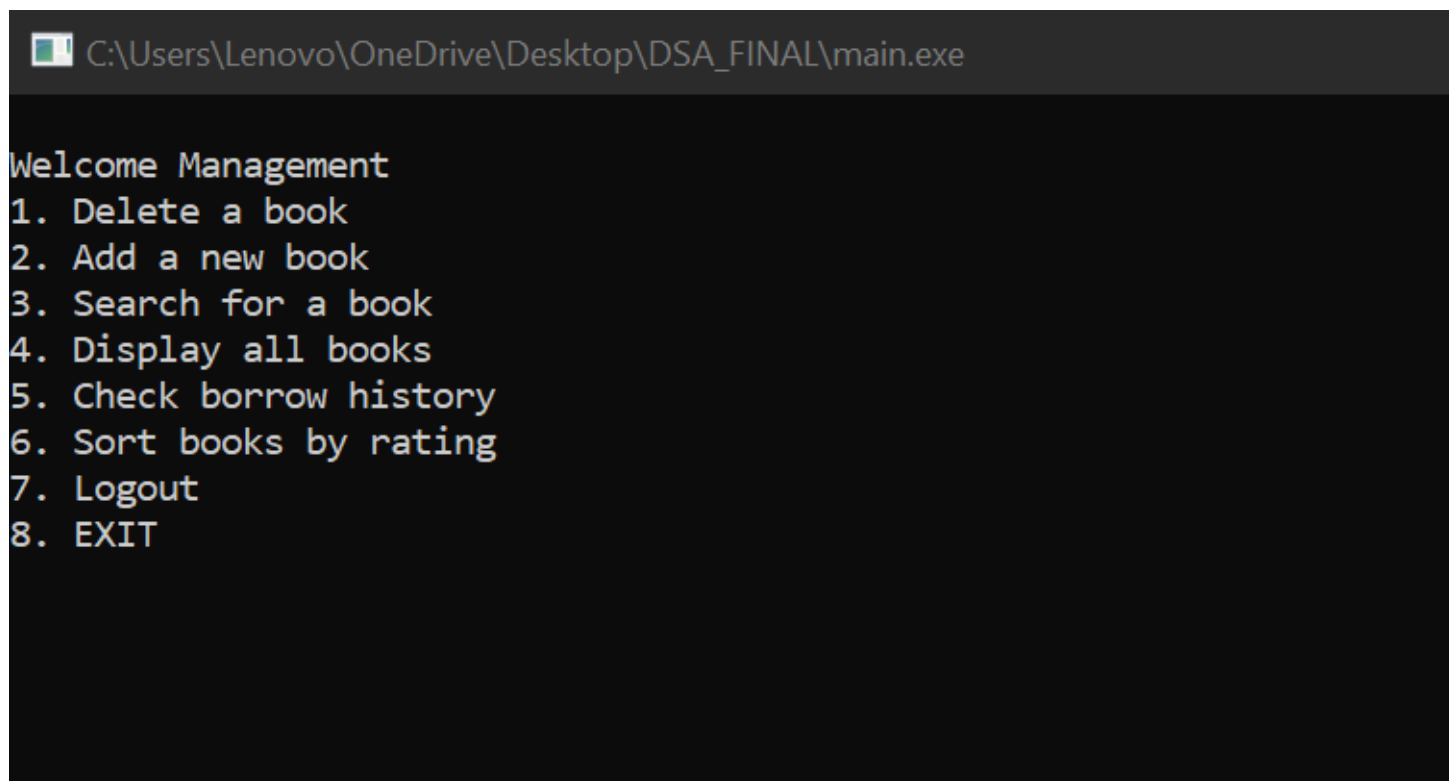
LOGIN INFORMATION:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****PLEASE ENTER YOUR LOGIN INFORMATION*****
Enter username:
Huzaifa
Enter password:
1234
```

A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe". The command prompt displays a login prompt: "*****PLEASE ENTER YOUR LOGIN INFORMATION*****". It then asks for a username, where "Huzaifa" is entered, and a password, where "1234" is entered.

Management Functionalities:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
Welcome Management
1. Delete a book
2. Add a new book
3. Search for a book
4. Display all books
5. Check borrow history
6. Sort books by rating
7. Logout
8. EXIT
```

A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe". The command prompt displays a welcome message: "Welcome Management". It then lists eight management functionalities: "1. Delete a book", "2. Add a new book", "3. Search for a book", "4. Display all books", "5. Check borrow history", "6. Sort books by rating", "7. Logout", and "8. EXIT".

Display all Books Function:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

Books in the Linked List:
1 . The Elements of Style
3 . The Meme Machine
5 . Help Your Kids with Computer Coding
7 . Computability and Unsolvability
9 . Coding Games in Scratch
11 . The Society of Mind
13 . The Connection Machine
15 . Introduction to Functional Programming
17 . Your First App: Node.js
19 . Seven Languages in Seven Weeks
21 . How Google Tests Software
23 . Pro Git
25 . Sams Teach Yourself MySQL in 24 Hours
27 . CSS: The Missing Manual
29 . Beginning Java 2
31 . Unity in Action
33 . Functional Programming in Scala
35 . Sams Teach Yourself Perl in 24 Hours
37 . "Programming in Lua Fourth Edition"
39 . Reviewing C++
2 . How Google Works
4 . Ship It!
6 . Game Programming Golden Rules
8 . Adventures in Minecraft
10 . Expert C Programming: Deep C Secrets
12 . JavaScript: The Good Parts
14 . Scrum and XP from the Trenches
16 . Chaos: Making a New Science
18 . Joel on Software
20 . Manage Your Project Portfolio
22 . Computability and Logic
24 . Best of Game Programming Gems
26 . Algorithms Unlocked
28 . Doing Data Science
30 . Learn You a Haskell for Great Good!
32 . The Little Schemer
34 . High Performance Browser Networking
36 . Learn Python The Hard Way
38 . A Tour of C++
40 . Game Programming Patterns
```

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

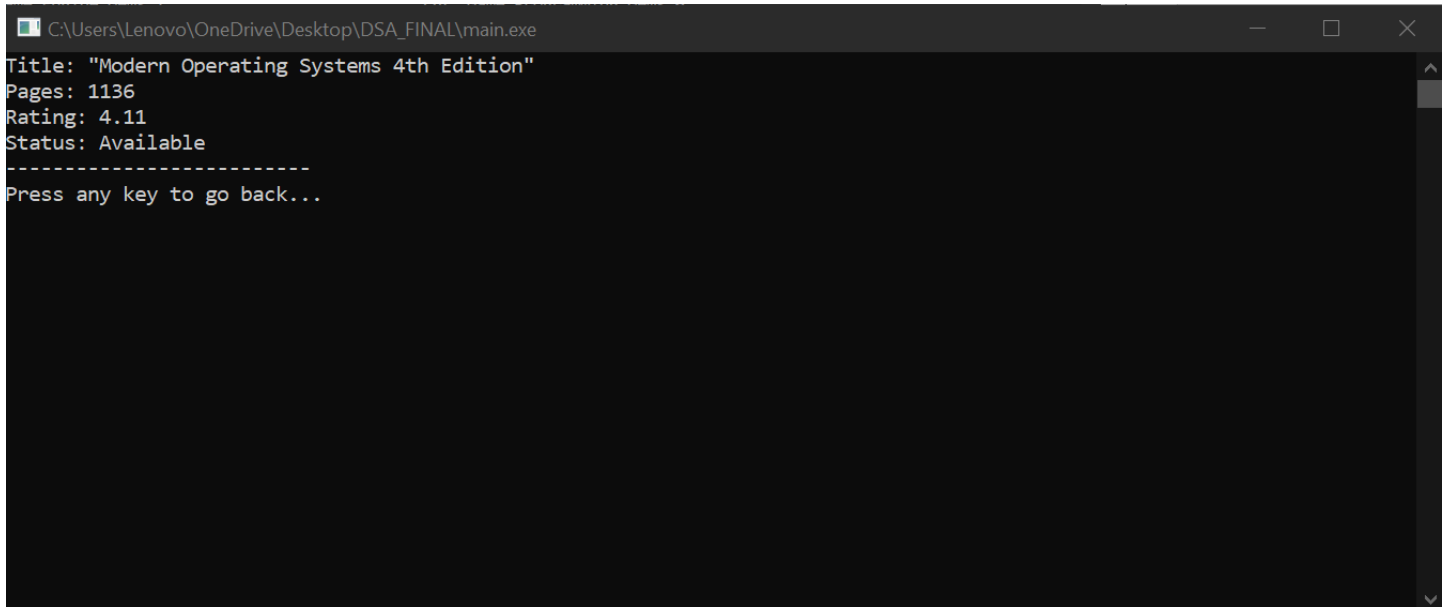
41 . From Mathematics to Generic Programming
43 . Learning Ruby
45 . Machine Learning for Hackers
47 . Making Games with Python & Pygame
49 . C# in Depth
51 . Real-World Machine Learning
53 . Unity Virtual Reality Projects
55 . Python for Data Analysis
57 . Practical Object Oriented Design in Ruby
59 . Building an FPS Game with Unity
61 . Make Your Own Neural Network
63 . The Joy of Clojure
65 . Quantum Computing Since Democritus
67 . Android NDK Game Development Cookbook
69 . Reviewing Java
71 . Basic Proof Theory
73 . Working Effectively with Legacy Code
75 . Object-Oriented Design Heuristics
77 . C++ Primer
79 . Dependency Injection in .NET
81 . Domain-Specific Languages
42 . Elements of Programming
44 . More Effective C++
46 . Code Complete
48 . Programming Pearls
50 . Learning Perl
52 . Managing Data Using Excel
54 . The Sciences of the Artificial
56 . Test Driven Development: By Example
58 . The Boost C++ Libraries
60 . The Psychology of Computer Programming
62 . The Rails Way
64 . Head First Design Patterns
66 . Data Structures and Algorithms
68 . Assembly Language: Step-By-Step
70 . Microsoft Visual C# Step by Step
72 . C: A Reference Manual
74 . API Design for C++
76 . C Primer Plus
78 . Mining of Massive Datasets
80 . Hacker's Delight
82 . Implementing Domain-Driven Design
```

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

83 . CLR via C# (Developer Reference)
85 . Programming PHP
87 . The C++ Programming Language
89 . The Unified Modeling Language User Guide
91 . Computational Complexity
93 . The UNIX Programming Environment
95 . Machine Learning
97 . Algorithm Design
99 . Engineering a Compiler
101 . Elements of the Theory of Computation
103 . Game Programming Gems 2
105 . Introduction to Algorithms
107 . Game Engine Architecture
109 . Geometric Tools for Computer Graphics
111 . "Modern Operating Systems 4th Edition"
113 . Game Engine Gems 1
115 . Game Programming Gems 4
117 . An Introduction to Database Systems
84 . Learning Python
86 . The C Programming Language
88 . Feynman Lectures On Computation
90 . Algorithms
92 . C++ Templates: The Complete Guide
94 . Inside the C++ Object Model
96 . The Algorithm Design Manual
98 . Deep Learning
100 . Database System Concepts
102 . Game Programming Gems
104 . Game Programming Gems 5
106 . Essentials of Programming Languages
108 . Computational Science and Engineering
110 . Modern Operating Systems
112 . Structured Computer Organization
114 . Game Programming Gems 6
116 . Game Programming Gems 3
118 . A Discipline for Software Engineering

Enter the number of the book to view details (0 to go back):
```

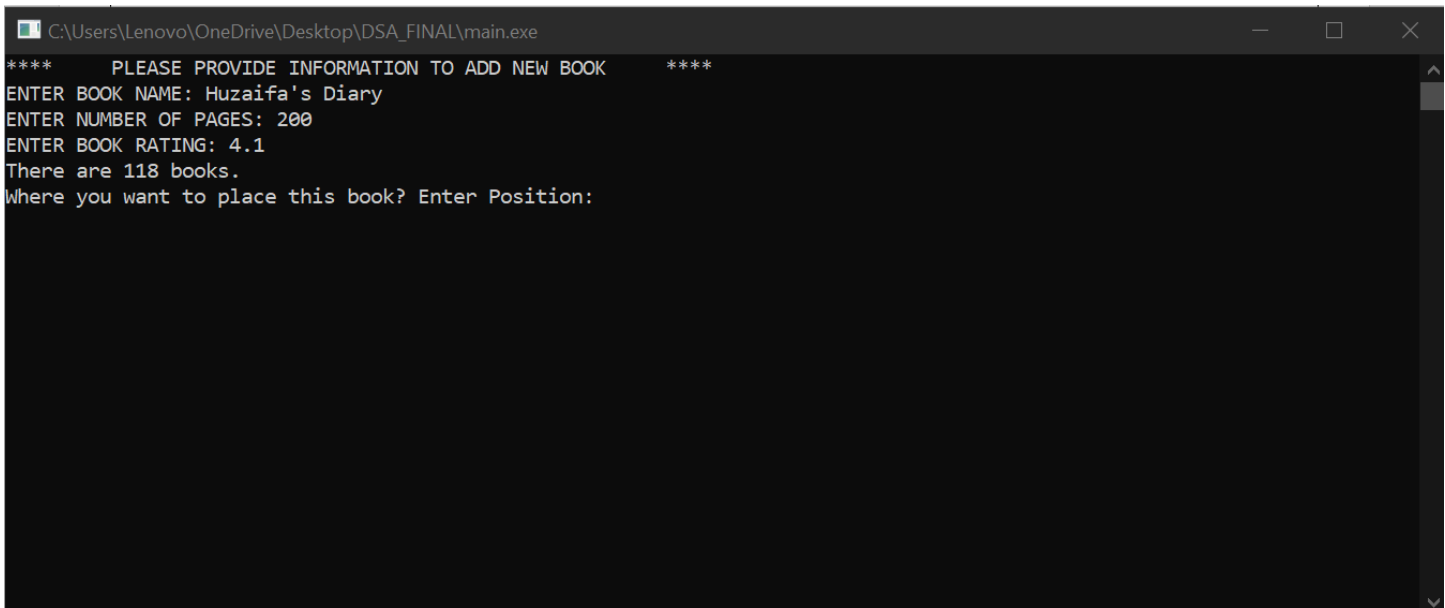
Book Detail Function:



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe". The command prompt displays the following text: "Title: 'Modern Operating Systems 4th Edition'", "Pages: 1136", "Rating: 4.11", "Status: Available", followed by a line of dashes "-----" and the prompt "Press any key to go back...".

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
Title: "Modern Operating Systems 4th Edition"
Pages: 1136
Rating: 4.11
Status: Available
-----
Press any key to go back...
```

Add new Book Function:



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe". The command prompt displays the following text: "**** PLEASE PROVIDE INFORMATION TO ADD NEW BOOK ****", "ENTER BOOK NAME: Huzaifa's Diary", "ENTER NUMBER OF PAGES: 200", "ENTER BOOK RATING: 4.1", "There are 118 books.", and the prompt "Where you want to place this book? Enter Position:". The cursor is positioned at the end of the last line.

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
**** PLEASE PROVIDE INFORMATION TO ADD NEW BOOK ****
ENTER BOOK NAME: Huzaifa's Diary
ENTER NUMBER OF PAGES: 200
ENTER BOOK RATING: 4.1
There are 118 books.
Where you want to place this book? Enter Position:
```

Message after adding Book:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****
"Huzaiifa's Diary" HAS BEEN ADDED SCCUSSFULLY!
PRESS ENTER KEY
```

Display Function after adding Book At Position 21:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
Books in the Linked List:
1 . The Elements of Style
2 . How Google Works
3 . The Meme Machine
4 . Ship It!
5 . Help Your Kids with Computer Coding
6 . Game Programming Golden Rules
7 . Computability and Unsolvability
8 . Adventures in Minecraft
9 . Coding Games in Scratch
10 . Expert C Programming: Deep C Secrets
11 . The Society of Mind
12 . JavaScript: The Good Parts
13 . The Connection Machine
14 . Scrum and XP from the Trenches
15 . Introduction to Functional Programming
16 . Chaos: Making a New Science
17 . Your First App: Node.js
18 . Joel on Software
19 . Seven Languages in Seven Weeks
20 . Manage Your Project Portfolio
21 . Huzaiifa's Diary
22 . How Google Tests Software
23 . Computability and Logic
24 . Pro Git
25 . Best of Game Programming Gems
26 . Sams Teach Yourself MySQL in 24 Hours
27 . Algorithms Unlocked
28 . CSS: The Missing Manual
29 . Doing Data Science
30 . Beginning Java 2
31 . Learn You a Haskell for Great Good!
32 . Unity in Action
33 . The Little Schemer
34 . Functional Programming in Scala
35 . High Performance Browser Networking
36 . Sams Teach Yourself Perl in 24 Hours
37 . Learn Python The Hard Way
38 . "Programming in Lua Fourth Edition"
39 . A Tour of C++
40 . Reviewing C++
```

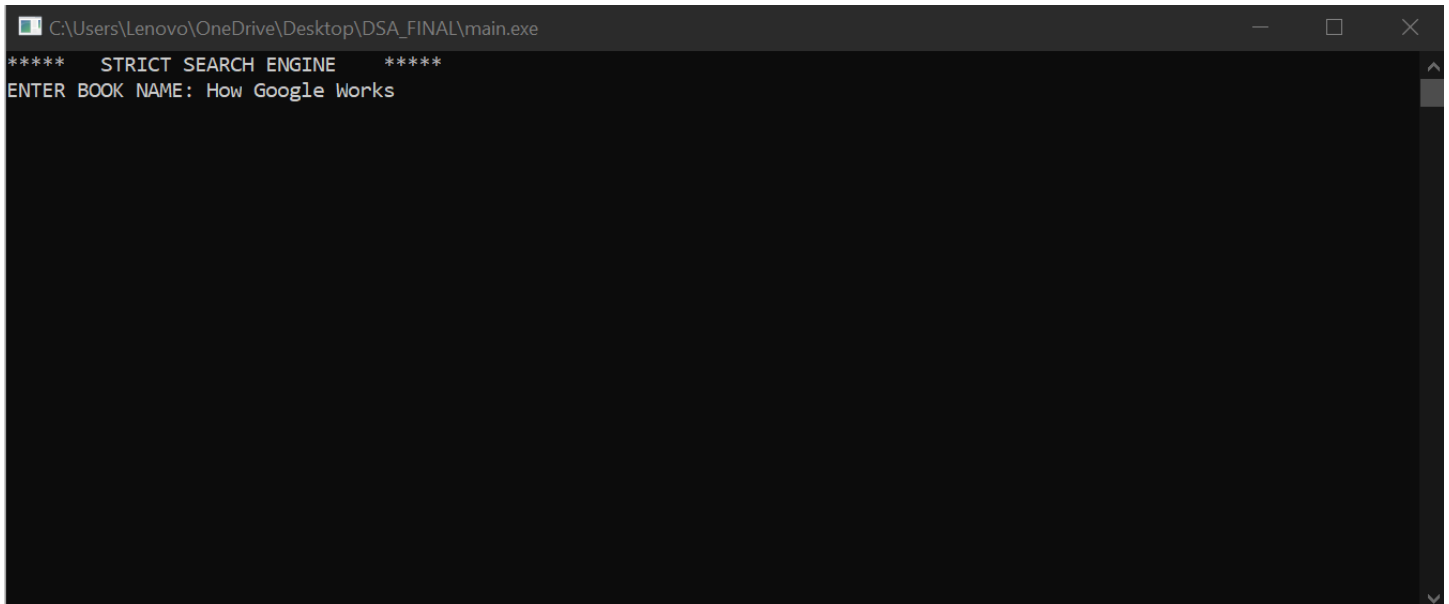
Delete a Book function first display all books then ask for a book to delete:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
81 . Hacker's Delight
83 . Implementing Domain-Driven Design
85 . Learning Python
87 . The C Programming Language
89 . Feynman Lectures On Computation
91 . Algorithms
93 . C++ Templates: The Complete Guide
95 . Inside the C++ Object Model
97 . The Algorithm Design Manual
99 . Deep Learning
101. Database System Concepts
103. Game Programming Gems
105. Game Programming Gems 5
107. Essentials of Programming Languages
109. Computational Science and Engineering
111. Modern Operating Systems
113. Structured Computer Organization
115. Game Programming Gems 6
117. Game Programming Gems 3
119. A Discipline for Software Engineering
82 . Domain-Specific Languages
84 . CLR via C# (Developer Reference)
86 . Programming PHP
88 . The C++ Programming Language
90 . The Unified Modeling Language User Guide
92 . Computational Complexity
94 . The UNIX Programming Environment
96 . Machine Learning
98 . Algorithm Design
100. Engineering a Compiler
102. Elements of the Theory of Computation
104. Game Programming Gems 2
106. Introduction to Algorithms
108. Game Engine Architecture
110. Geometric Tools for Computer Graphics
112. "Modern Operating Systems 4th Edition"
114. Game Engine Gems 1
116. Game Programming Gems 4
118. An Introduction to Database Systems
Which book do you want to delete? Enter Position:
```

Deletion Message:

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****
"Huzaiifa's Diary" HAS BEEN DELETED SCCUSSFULLY!
PRESS ENTER KEY
```

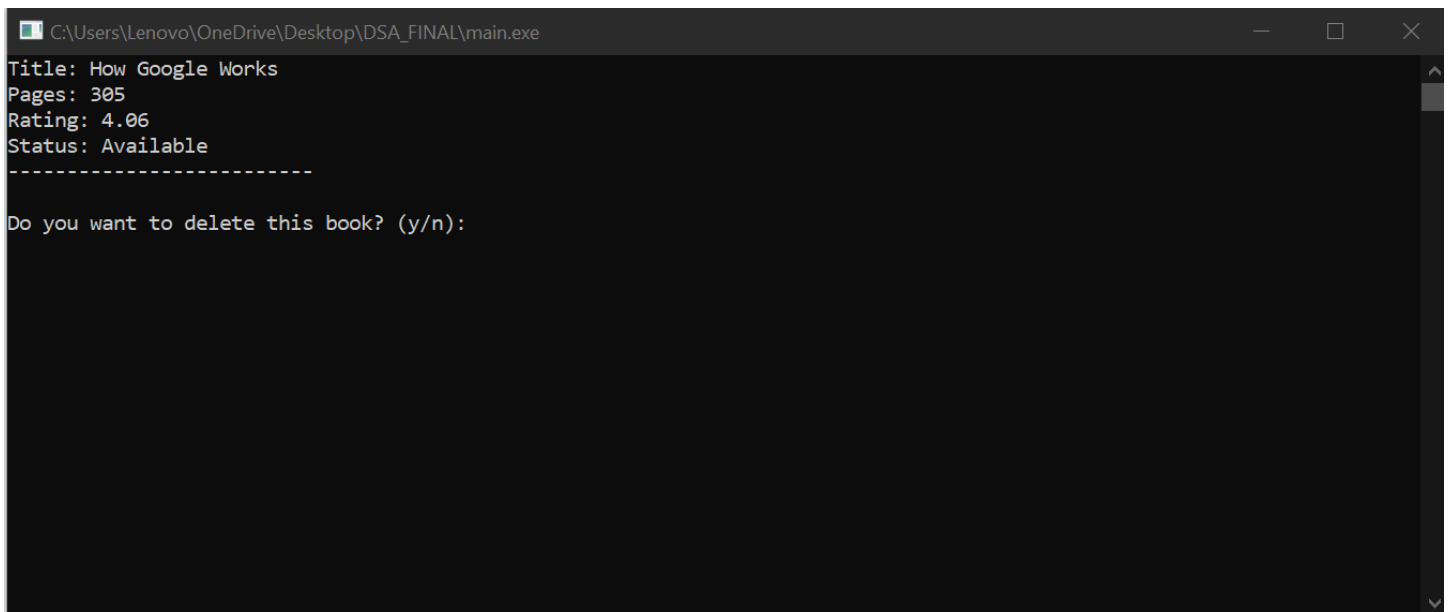
Search Function:



A screenshot of a Windows command prompt window. The title bar shows the file path: C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe. The window contains the following text: ***** STRICT SEARCH ENGINE ***** followed by ENTER BOOK NAME: How Google Works. The rest of the window is empty.

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****  STRICT SEARCH ENGINE  *****
ENTER BOOK NAME: How Google Works
```

Search Function Output Message:

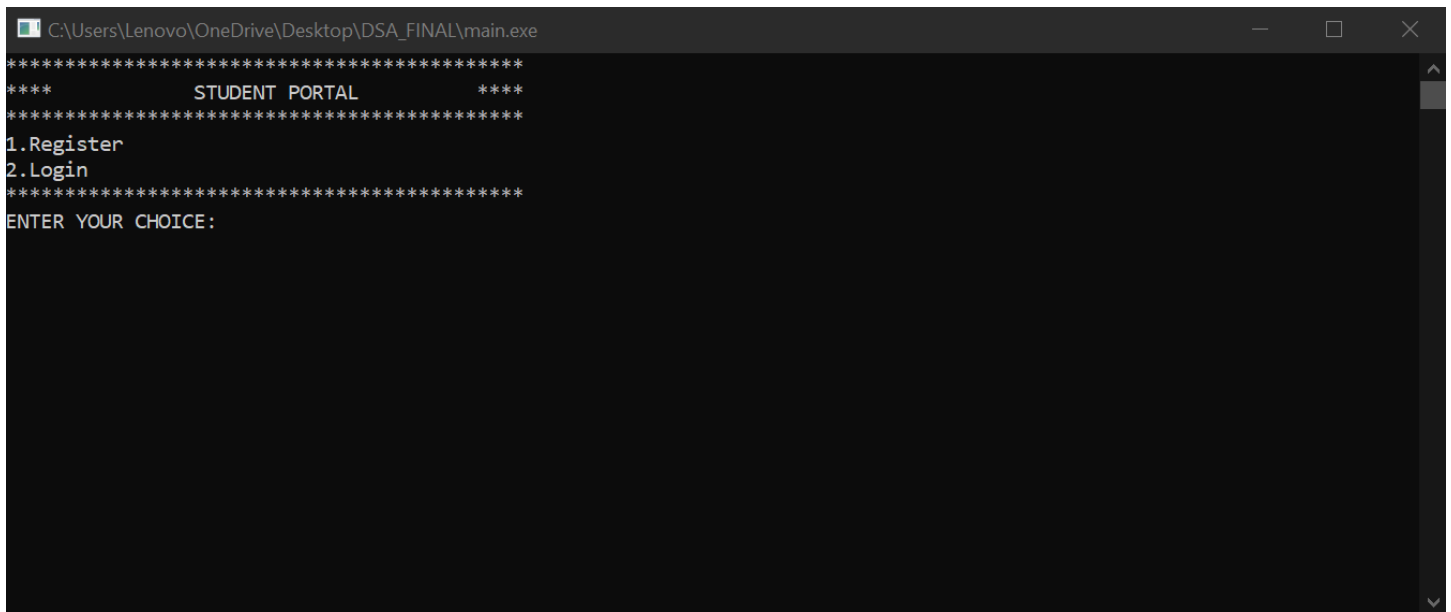


A screenshot of a Windows command prompt window. The title bar shows the file path: C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe. The window displays the output of the search function for the book 'How Google Works'. The output includes the title, page count, rating, and status, followed by a separator line and a prompt to delete the book.

```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
Title: How Google Works
Pages: 305
Rating: 4.06
Status: Available
-----
Do you want to delete this book? (y/n):
```

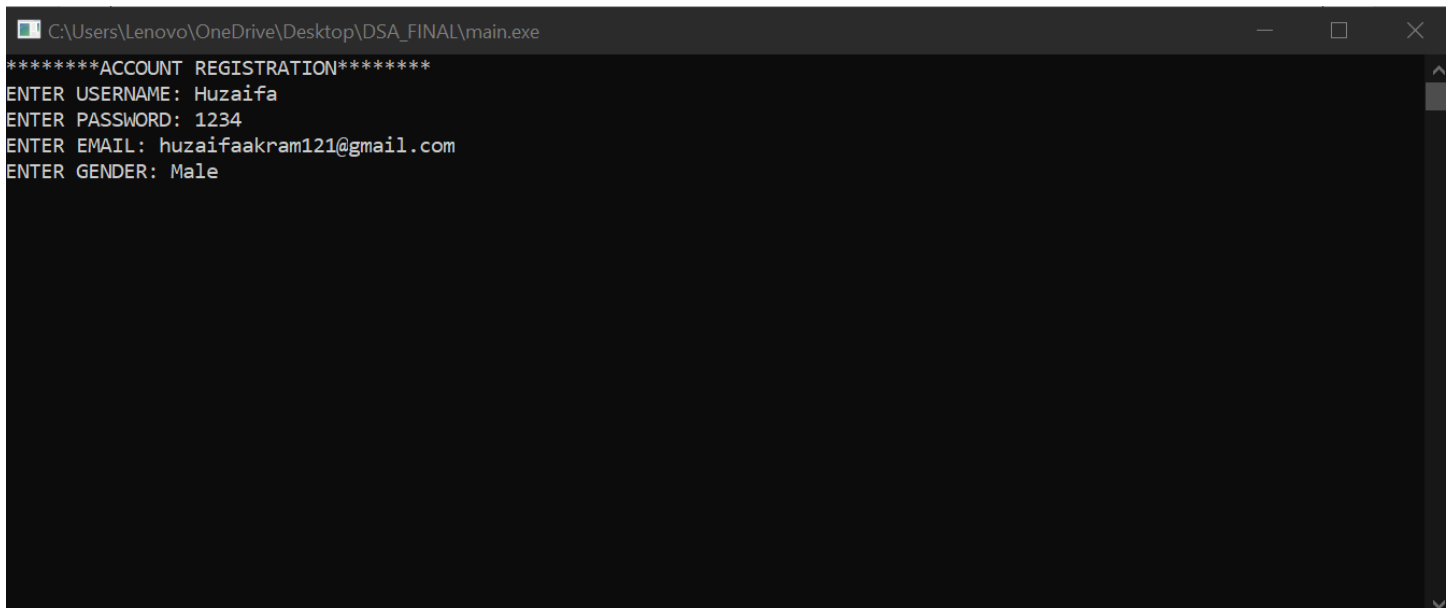
STUDENT PORTAL:

LOGIN OR REGISTER MESSAGE:



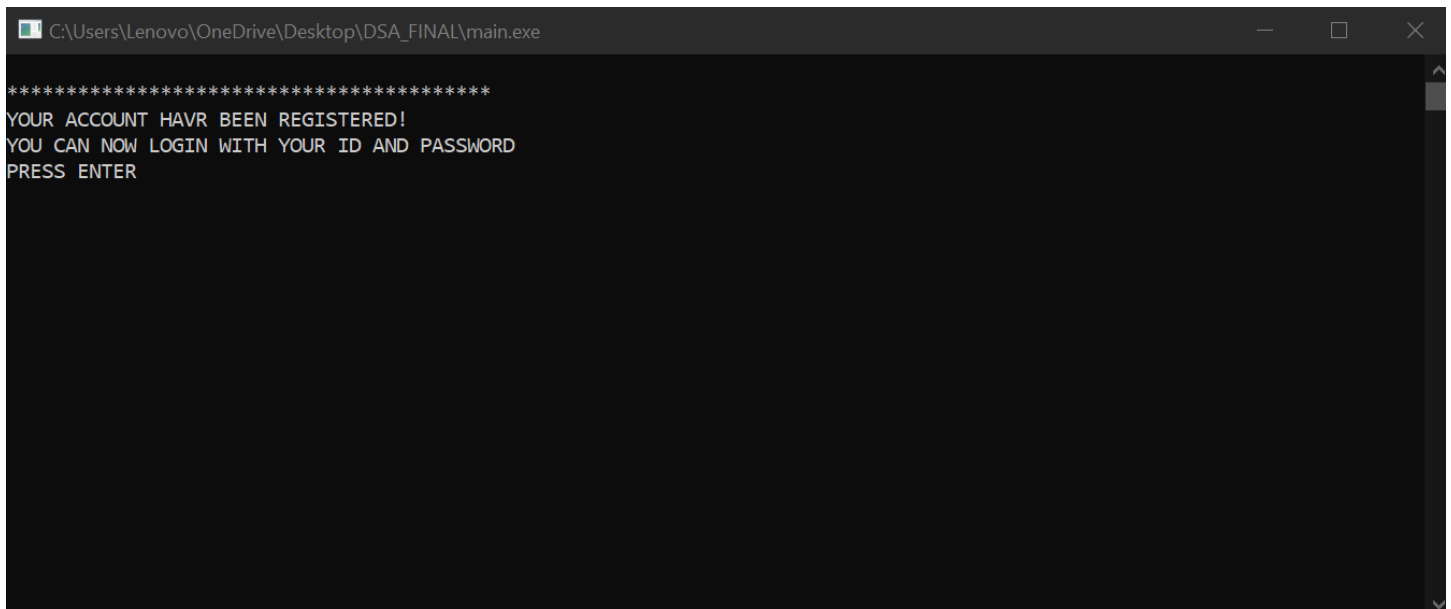
```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****
***      STUDENT PORTAL      ***
*****
1.Register
2.Login
*****
ENTER YOUR CHOICE:
```

Account Registration Function:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****ACCOUNT REGISTRATION*****
ENTER USERNAME: Huzaiifa
ENTER PASSWORD: 1234
ENTER EMAIL: huzaiifaakram121@gmail.com
ENTER GENDER: Male
```

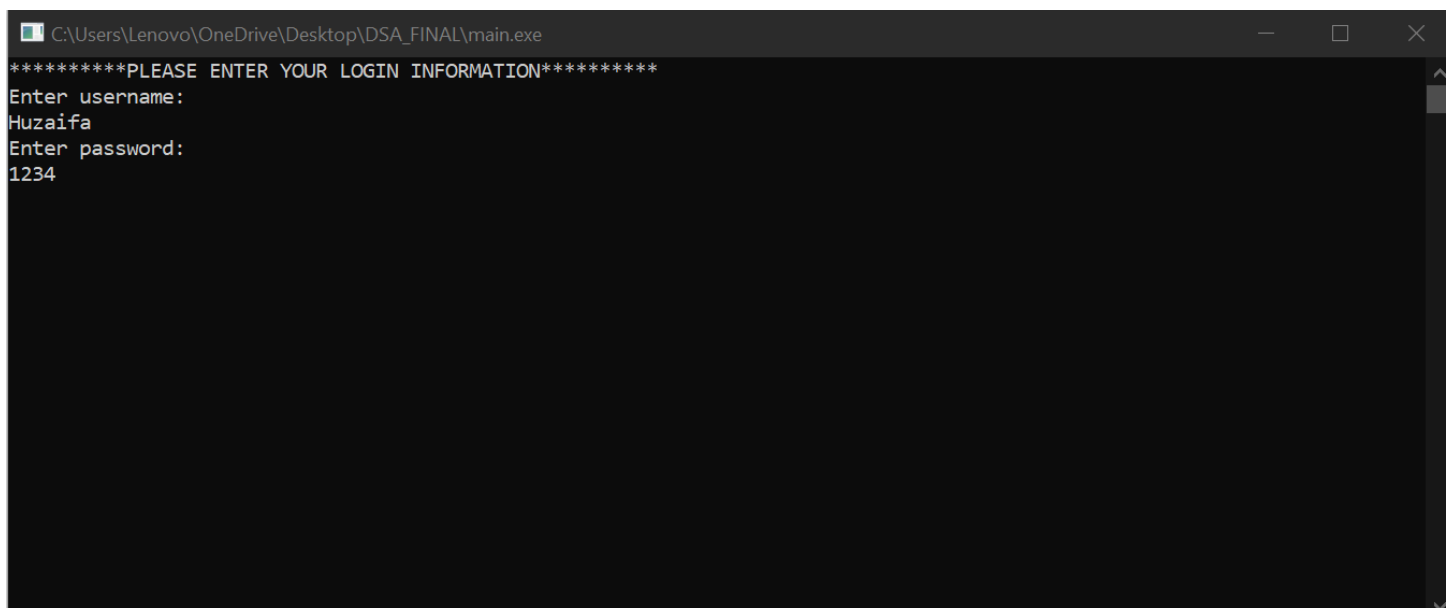
Registration Successful Message:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

*****
YOUR ACCOUNT HAVR BEEN REGISTERED!
YOU CAN NOW LOGIN WITH YOUR ID AND PASSWORD
PRESS ENTER
```

Login using Registered Name and Password:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

*****PLEASE ENTER YOUR LOGIN INFORMATION*****
Enter username:
Huzaifa
Enter password:
1234
```

Student Functionalities:

```
Select C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

Welcome Student
1. Borrow a book
2. Return a book
3. Search a book
4. Display all books
5. Logout
6. EXIT
```

Borrow Book Function first display all books then ask for book which user want to borrow:

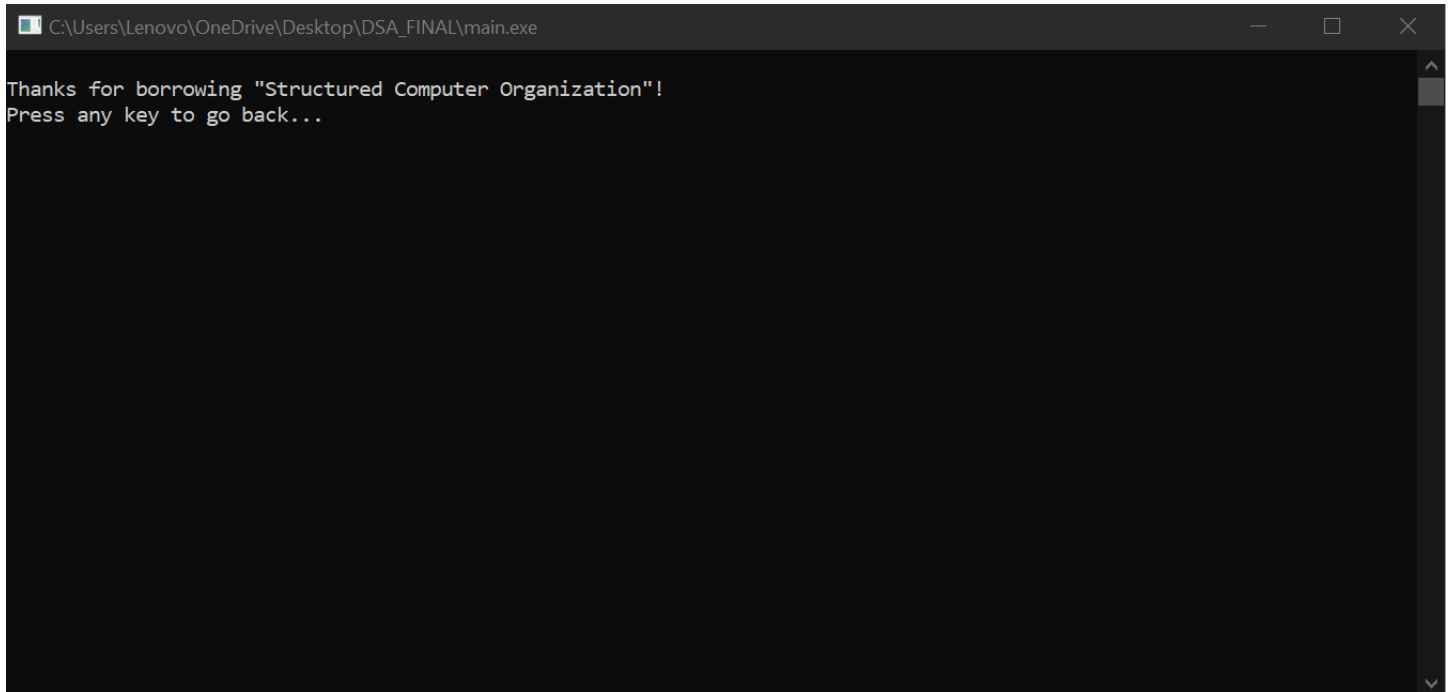
```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe

73 . Working Effectively with Legacy Code
75 . Object-Oriented Design Heuristics
77 . C++ Primer
79 . Dependency Injection in .NET
81 . Domain-Specific Languages
83 . CLR via C# (Developer Reference)
85 . Programming PHP
87 . The C++ Programming Language
89 . The Unified Modeling Language User Guide
91 . Computational Complexity
93 . The UNIX Programming Environment
95 . Machine Learning
97 . Algorithm Design
99 . Engineering a Compiler
101. Elements of the Theory of Computation
103. Game Programming Gems 2
105. Introduction to Algorithms
107. Game Engine Architecture
109. Geometric Tools for Computer Graphics
111. "Modern Operating Systems 4th Edition"
113. Game Engine Gems 1
115. Game Programming Gems 4
117. An Introduction to Database Systems

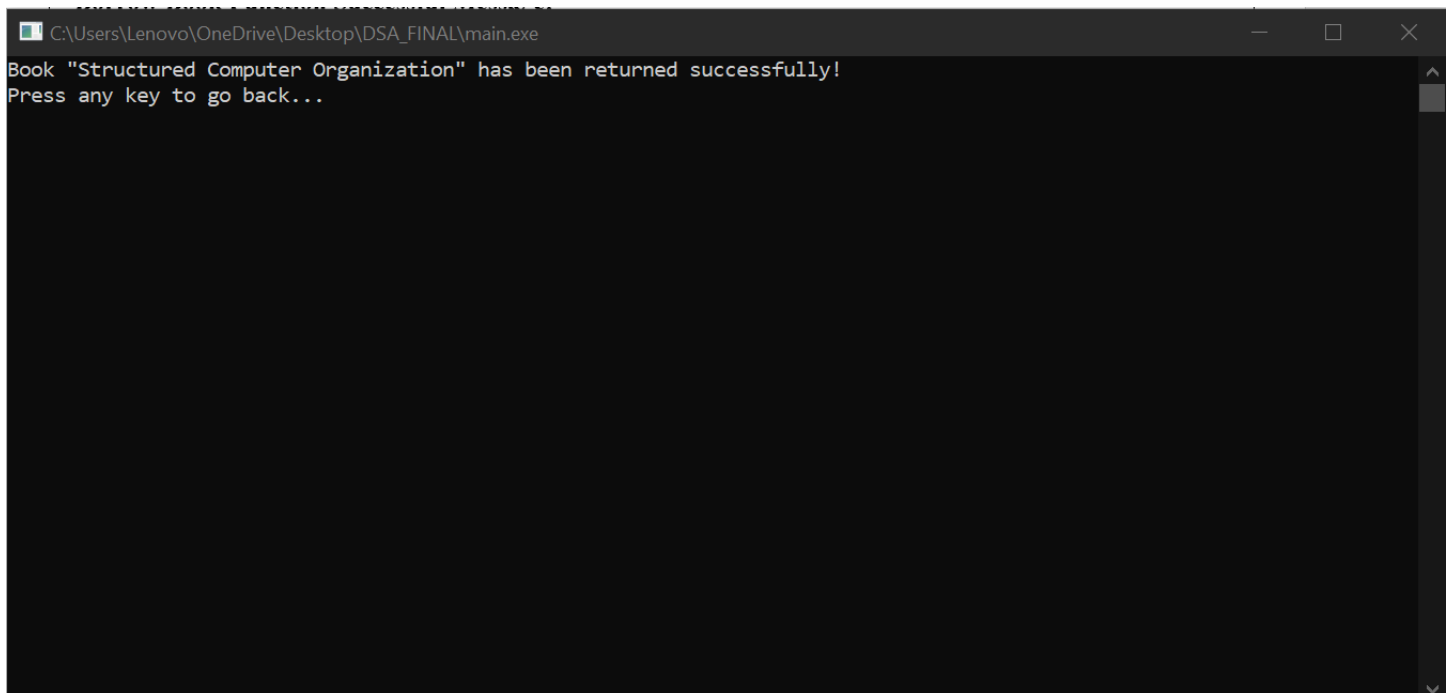
74 . API Design for C++
76 . C Primer Plus
78 . Mining of Massive Datasets
80 . Hacker's Delight
82 . Implementing Domain-Driven Design
84 . Learning Python
86 . The C Programming Language
88 . Feynman Lectures On Computation
90 . Algorithms
92 . C++ Templates: The Complete Guide
94 . Inside the C++ Object Model
96 . The Algorithm Design Manual
98 . Deep Learning
100. Database System Concepts
102. Game Programming Gems
104. Game Programming Gems 5
106. Essentials of Programming Languages
108. Computational Science and Engineering
110. Modern Operating Systems
112. Structured Computer Organization
114. Game Programming Gems 6
116. Game Programming Gems 3
118. A Discipline for Software Engineering

Enter the number of the book to borrow (0 to go back):
```

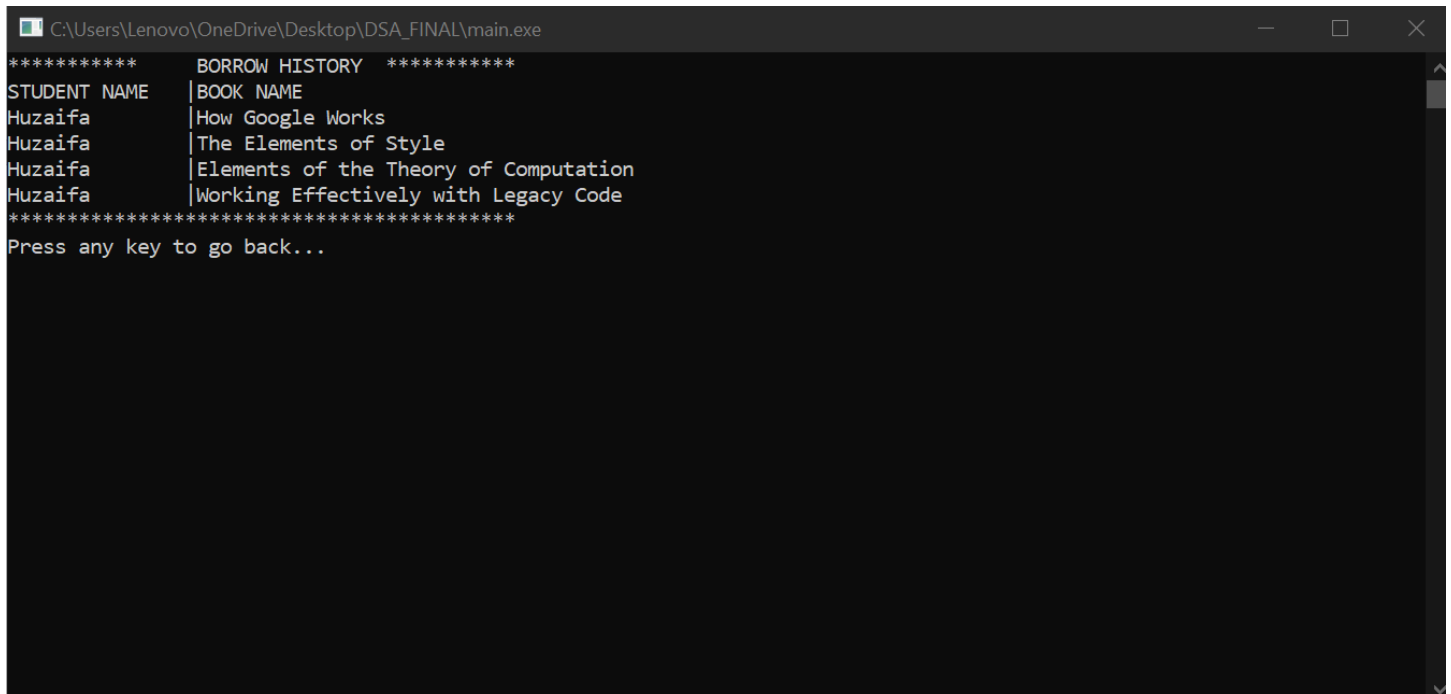
Borrow Book Function Successful Message:



Return Book Function:

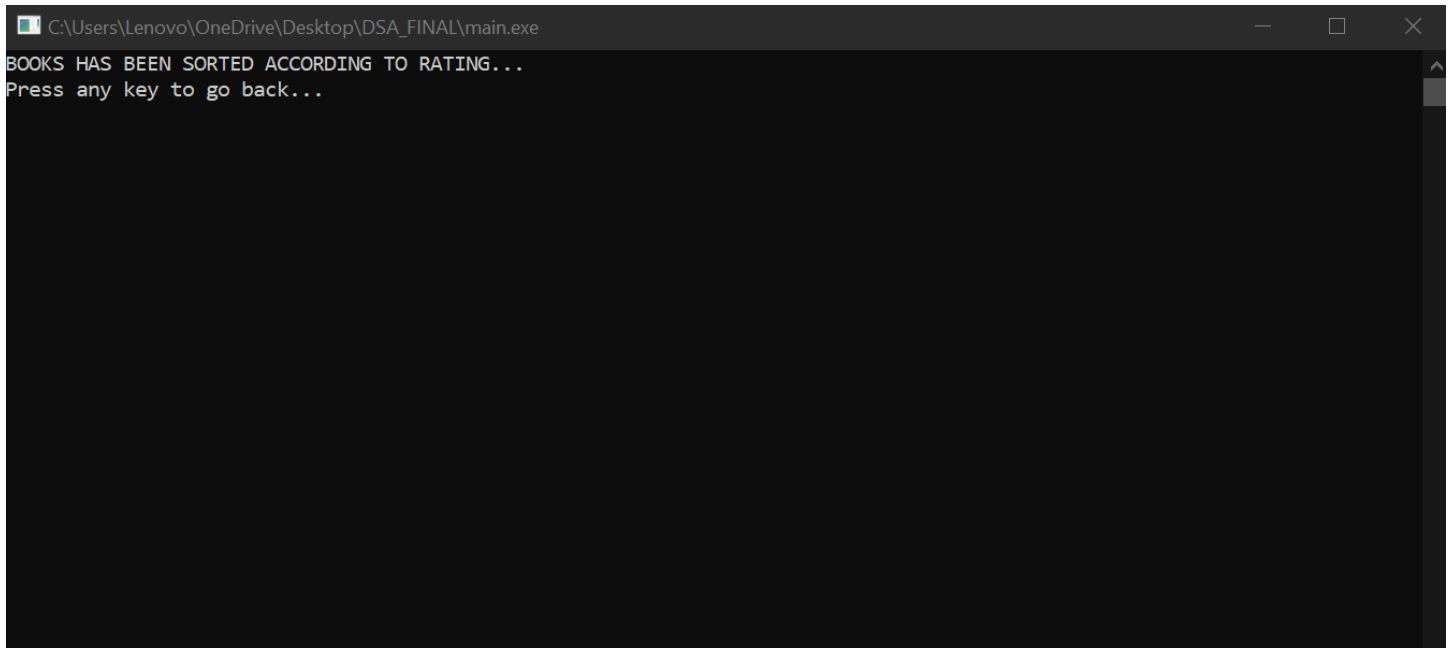


Management Function of Check Borrow History:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
*****      BORROW HISTORY      *****
STUDENT NAME | BOOK NAME
Huzaifa     | How Google Works
Huzaifa     | The Elements of Style
Huzaifa     | Elements of the Theory of Computation
Huzaifa     | Working Effectively with Legacy Code
*****
Press any key to go back...
```

Management Function of Selection Sort:



```
C:\Users\Lenovo\OneDrive\Desktop\DSA_FINAL\main.exe
BOOKS HAS BEEN SORTED ACCORDING TO RATING...
Press any key to go back...
```